

Assignment #2

2026-06-07

R Markdown

```
library(caret)
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```
library(class)
```

```
library(dplyr)
```

```
##
```

```
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
## filter, lag
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
## intersect, setdiff, setequal, union
```

```
##Question 1 I am loading the dataset and making sure its the correct dataset using the head function. I also name the dataset mydata.
```

```
mydata <- read.csv("UniversalBank-1.csv")
```

```
head(mydata)
```

```
##   ID Age Experience Income ZIP.Code Family CCAvg Education Mortgage
## 1  1  25           1     49   91107      4   1.6           1         0
## 2  2  45          19     34   90089      3   1.5           1         0
## 3  3  39          15     11   94720      1   1.0           1         0
## 4  4  35           9    100   94112      1   2.7           2         0
## 5  5  35           8     45   91330      4   1.0           2         0
## 6  6  37          13     29   92121      4   0.4           2        155
##   Personal.Loan Securities.Account CD.Account Online CreditCard
## 1              0                  1           0         0         0
## 2              0                  1           0         0         0
## 3              0                  0           0         0         0
## 4              0                  0           0         0         0
## 5              0                  0           0         0         1
## 6              0                  0           0         1         0
```

Here, I am using the str function to look at the structure of my data.

```
str(mydata)
```

```
## 'data.frame':    5000 obs. of  14 variables:
## $ ID             : int  1 2 3 4 5 6 7 8 9 10 ...
## $ Age             : int  25 45 39 35 35 37 53 50 35 34 ...
## $ Experience       : int  1 19 15 9 8 13 27 24 10 9 ...
## $ Income           : int  49 34 11 100 45 29 72 22 81 180 ...
## $ ZIP.Code         : int  91107 90089 94720 94112 91330 92121 91711 93943 90089 93023 ...
## $ Family           : int  4 3 1 1 4 4 2 1 3 1 ...
## $ CCAvg            : num  1.6 1.5 1 2.7 1 0.4 1.5 0.3 0.6 8.9 ...
## $ Education        : int  1 1 1 2 2 2 2 3 2 3 ...
## $ Mortgage         : int  0 0 0 0 0 155 0 0 104 0 ...
## $ Personal.Loan    : int  0 0 0 0 0 0 0 0 0 1 ...
## $ Securities.Account : int  1 1 0 0 0 0 0 0 0 0 ...
## $ CD.Account       : int  0 0 0 0 0 0 0 0 0 0 ...
## $ Online           : int  0 0 0 0 0 1 1 0 1 0 ...
## $ CreditCard       : int  0 0 0 0 1 0 0 1 0 0 ...
```

Education needs to be transformed into a dummy variable because the distances between 1,2,3 mean nothing. Education is categorical and it must be recognized like this when doing the modeling.

```
mydata$Education <- as.factor(mydata$Education)
dummy_model <- dummyVars(~Education,data=mydata)
education_dummies <- predict(dummy_model,mydata)
head(education_dummies)
```

```
##   Education.1 Education.2 Education.3
## 1           1           0           0
## 2           1           0           0
## 3           1           0           0
## 4           0           1           0
## 5           0           1           0
## 6           0           1           0
```

Now that I turned education into a dummy variable I have to combine it with the dataset

```
mydata <- cbind(mydata,education_dummies)
```

Now I have to remove the original education column

```
mydata$Education <- NULL
```

Checking my columns to see if it worked

```
names(mydata)
```

```
## [1] "ID"           "Age"          "Experience"
## [4] "Income"       "ZIP.Code"     "Family"
## [7] "CAvg"        "Mortgage"     "Personal.Loan"
## [10] "Securities.Account" "CD.Account"   "Online"
## [13] "CreditCard"   "Education.1"  "Education.2"
## [16] "Education.3"
```

Now that I have successfully transformed education into a dummy variable I am now going to remove ID and zip code from the dataset.

```
mydata$ID <- NULL
mydata$ZIP.Code <- NULL
names(mydata)
```

```
## [1] "Age"           "Experience"      "Income"
## [4] "Family"        "CCAvg"          "Mortgage"
## [7] "Personal.Loan" "Securities.Account" "CD.Account"
## [10] "Online"        "CreditCard"     "Education.1"
## [13] "Education.2"    "Education.3"
```

Next we need to normalize the data. We do this because the original variables have different scales, some larger than others.

I am now going to normalize the predictors

```
predictors <- mydata[,names(mydata)!="Personal.Loan"]
norm.values <- preprocess(predictors,method="range")
predictors_norm <- predict(norm.values,predictors)
mydata_norm <- cbind(predictors_norm,Personal.Loan=mydata$Personal.Loan)
```

Split into training and validation sets

```
set.seed(1)
train.index <- createDataPartition(mydata_norm$Personal.Loan,p=.60,list=FALSE)
trainData <- mydata_norm[train.index,]
validData <- mydata_norm[-train.index,]
```

Separate predictors and outcome

```
train_x <- trainData[,names(trainData)!="Personal.Loan"]
valid_x <- validData[,names(validData)!="Personal.Loan"]
train_y <- trainData$Personal.Loan
valid_y <- validData$Personal.Loan
```

Add new customer information as a dataframe

```
new_customer <- data.frame(Age=40,Experience=10,Income=84,Family=2,CCAvg=2,Mortgage=0,Securities.Account=0,Online=0,CreditCard=0,Education.1=0,Education.2=0,Education.3=0)
new_customer_norm <- predict(norm.values,new_customer)
knn_pred_k1 <- knn(train=train_x,test=new_customer_norm,cl=train_y,k=1)
knn_pred_k1
```

```
## [1] 0
## Levels: 0 1
```

The new customer classified as 0 with k=1, meaning the customer will not accept the personal loan offer according to the model's prediction.

Question 2 In this question I need to find the k that balances between overfitting and underfitting. I am to create an empty list to store the accuracy results for each value of K (1-20). Then I am going to output the max accuracy for a certain value of K.

```

accuracy <- c()
for(i in 1:20){pred <- knn(train=train_x,test=valid_x,cl=train_y,k=i)
accuracy[i] <- mean(pred==valid_y)}
accuracy

## [1] 0.9610 0.9515 0.9580 0.9560 0.9530 0.9490 0.9475 0.9435 0.9460 0.9440
## [11] 0.9395 0.9395 0.9375 0.9295 0.9270 0.9250 0.9240 0.9225 0.9225 0.9210

```

```
which.max(accuracy)
```

```
## [1] 1
```

```
max(accuracy)
```

```
## [1] 0.961
```

After evaluating values of k based on validation accuracy, k=1 had the highest accuracy 96.1%, therefore, making it the best value of k.

##Question 3

We know that the best model to use is one where k=1. Lets run the model on the validation data, then analyze the performance of the model using a confusion matrix.

```

best_pred <- knn(train=train_x,test=valid_x,cl=train_y,k=1)
confusionMatrix(best_pred,as.factor(valid_y))

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 1770   53
##           1   25  152
##
##              Accuracy : 0.961
##              95% CI : (0.9516, 0.9691)
##      No Information Rate : 0.8975
##      P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.7744
##
##  McNemar's Test P-Value : 0.002235
##
##              Sensitivity : 0.9861
##              Specificity : 0.7415
##              Pos Pred Value : 0.9709
##              Neg Pred Value : 0.8588
##              Prevalence : 0.8975
##              Detection Rate : 0.8850
##      Detection Prevalence : 0.9115
##              Balanced Accuracy : 0.8638
##
##              'Positive' Class : 0
##

```

This predictive model has an 96.1% overall accuracy. The confusion matrix showed 1,770 true negatives, 152 true positives, 25 false positives, and 53 false negatives.

##Question 4 This question is basically asking us to classify the same customer we did in the first part of the problem using the best k. We already know the best k=1, so the customer is classified as 0.

##Question 5

First we have to create a new split. The overall data is going to be split into training data, validation data, and test data. 50%, 30%, and 20% respectively.

2500 customers need to go to training

```
set.seed(1)
train.index <- createDataPartition(mydata_norm$Personal.Loan,p=.50,list=FALSE)
trainData2 <- mydata_norm[train.index,]
remainingData <- mydata_norm[-train.index,]
```

Now we split the remaining data

```
valid.index <- createDataPartition(remainingData$Personal.Loan,p=.60,list=FALSE)
validData2 <- remainingData[valid.index,]
testData <- remainingData[-valid.index,]
```

To see if the sizes are correct we check

```
dim(trainData2)
```

```
## [1] 2500  14
```

```
dim(validData2)
```

```
## [1] 1500  14
```

```
dim(testData)
```

```
## [1] 1000  14
```

X and Y must be created

```
train_x2 <- trainData2[,names(trainData2)!="Personal.Loan"]
valid_x2 <- validData2[,names(validData2)!="Personal.Loan"]
test_x <- testData[,names(testData)!="Personal.Loan"]
train_y2 <- trainData2$Personal.Loan
valid_y2 <- validData2$Personal.Loan
test_y <- testData$Personal.Loan
```

Now using the k=1 prediction model we are going to make predictions for training set, validation set, and test set

```

pred_train <- knn(train=train_x2,test=train_x2,cl=train_y2,k=1)
pred_valid <- knn(train=train_x2,test=valid_x2,cl=train_y2,k=1)
pred_test <- knn(train=train_x2,test=test_x,cl=train_y2,k=1)

```

Confusion matrixes for all three

```

confusionMatrix(pred_train,as.factor(train_y2))

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 2268    0
##           1    0  232
##
##           Accuracy : 1
##           95% CI : (0.9985, 1)
##       No Information Rate : 0.9072
##       P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 1
##
##  Mcnemar's Test P-Value : NA
##
##       Sensitivity : 1.0000
##       Specificity : 1.0000
##       Pos Pred Value : 1.0000
##       Neg Pred Value : 1.0000
##       Prevalence : 0.9072
##       Detection Rate : 0.9072
##       Detection Prevalence : 0.9072
##       Balanced Accuracy : 1.0000
##
##       'Positive' Class : 0
##

```

```

confusionMatrix(pred_valid,as.factor(valid_y2))

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 1341   39
##           1   23   97
##
##           Accuracy : 0.9587
##           95% CI : (0.9473, 0.9682)
##       No Information Rate : 0.9093
##       P-Value [Acc > NIR] : 1.294e-13
##
##           Kappa : 0.7353
##

```

```
## McNemar's Test P-Value : 0.05678
##
##      Sensitivity : 0.9831
##      Specificity : 0.7132
##      Pos Pred Value : 0.9717
##      Neg Pred Value : 0.8083
##      Prevalence : 0.9093
##      Detection Rate : 0.8940
##      Detection Prevalence : 0.9200
##      Balanced Accuracy : 0.8482
##
##      'Positive' Class : 0
##
```

```
confusionMatrix(pred_test,as.factor(test_y))
```

```
## Confusion Matrix and Statistics
##
##      Reference
## Prediction  0   1
##      0  874  25
##      1   14  87
##
##      Accuracy : 0.961
##      95% CI : (0.9471, 0.9721)
##      No Information Rate : 0.888
##      P-Value [Acc > NIR] : <2e-16
##
##      Kappa : 0.7951
##
## McNemar's Test P-Value : 0.1093
##
##      Sensitivity : 0.9842
##      Specificity : 0.7768
##      Pos Pred Value : 0.9722
##      Neg Pred Value : 0.8614
##      Prevalence : 0.8880
##      Detection Rate : 0.8740
##      Detection Prevalence : 0.8990
##      Balanced Accuracy : 0.8805
##
##      'Positive' Class : 0
##
```

After splitting the normalized data into training 50%, validation 30%, and test 20% I ran the model. The training data achieved 100% accuracy. The validation set received 95.87%. And the test set received 96.10% accuracy. This shows that the model effectively memorized the training set. The results also indicate that the model generalizes well to unseen data.